

ICS 1201 - Data Structures and Algorithms

Breadth-First Search (BFS) & Depth-First Search (DFS)
Lab Reports

GROUP MEMBERS:

Mark Nyanjui	– 216480
Joseph Kihara	– 180785
Jemeli Nazlin	– 220546
Imani Mathenge	– 220687
Andre Ryan	– 221123

Document Overview

This document contains the cleaned student lab reports for both the Breadth-First Search (BFS) and Depth-First Search (DFS) exercises. The presentation and structure have been aligned seamlessly to reflect a collective team effort.

Sections included in each report:

- Introduction
- Graph used in the lab
- Manual trace
- Before and after code changes
- Explanation of the changes
- Output screenshots & text transcriptions
- Discussion and conclusion

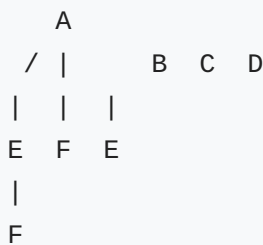
BREADTH-FIRST SEARCH (BFS) LAB REPORT

1. Introduction

In this lab, we worked with Breadth-First Search (BFS) and used the given graph to understand how BFS moves through nodes. We learned that BFS starts from one node and visits all the closest neighbors first before moving to nodes that are further away. This means BFS explores the graph level by level using a queue.

2. Graph Used in the Lab

The graph used in the BFS lab can be drawn in a simple way with arrows showing the direction of each edge:



Adjacency list: $A \rightarrow [B, C, D]$, $B \rightarrow [E]$, $C \rightarrow [F]$, $D \rightarrow [E]$, $E \rightarrow [F]$, $F \rightarrow []$

3. Manual BFS Trace

Starting from A, BFS first visits A, then all of A's direct neighbors, which are B, C, and D. After finishing those, it moves to the next level and visits E and F. So the manual BFS trace is:

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$$

This means the final BFS visit order is: **[A, B, C, D, E, F]**.

4. Before and After Code Changes

Before the exercise changes, the graph ended at F and the code performed the normal BFS traversal, distance calculation, and path checking.

```
graph = {
    'A': ['B', 'C', 'D'],
    'B': ['E'],
    'C': ['F'],
    'D': ['E'],
    'E': ['F'],
}
```

```
'F': []
}
```

After the exercise changes, we added node G with an edge $F \rightarrow G$. We also changed the path function so that it can return the actual path, and we grouped nodes by BFS levels.

```
graph = {
    'A': ['B', 'C', 'D'],
    'B': ['E'],
    'C': ['F'],
    'D': ['E'],
    'E': ['F'],
    'F': ['G'],
    'G': []
}
# bfs_path_exists was updated to return the actual path
# bfs_with_distance was also extended to group nodes by level
```

5. Explanation of the Changes

The first change was adding node G after F. This made the graph one level deeper, so when BFS reaches F it can continue to G. The second change was improving the path function so that instead of only saying whether a path exists, it can also print the actual path between two nodes. The third change was grouping the nodes by levels, which makes it easier to see how BFS works level by level.

6. Before Output Screenshot

This is the output from the original BFS program before the exercise modifications were added.

```
→ Added to queue: ['E']
Step 5 Queue contents: ['E', 'F']
Visiting 'E' | Path so far: ['A', 'B', 'C', 'D', 'E']
→ Added to queue: ['F']
Step 6 Queue contents: ['F', 'E', 'F']
Visiting 'F' | Path so far: ['A', 'B', 'C', 'D', 'E', 'F']
→ No new neighbours to add
Step 7 Queue contents: ['E', 'F']
Step 8 Queue contents: ['F']

BFS complete! Final order: ['A', 'B', 'C', 'D', 'E', 'F']
=====
BFS with distances from A:
A: distance = 0
B: distance = 1
C: distance = 1
```

```
D: distance = 1
E: distance = 2
F: distance = 2

Path existence checks:
A → F: Path EXISTS
A → E: Path EXISTS
F → A: X No path
C → B: X No path
```

7. After Output Screenshot

This is the output after we made the required changes. It now includes node G, actual paths, and BFS levels.

```
[3] BFS with distances from A:
A: distance = 0
B: distance = 1
C: distance = 1
D: distance = 1
E: distance = 2
F: distance = 2
G: distance = 3

[4] Path existence checks:
A → F: Path EXISTS
A → E: Path EXISTS
F → A: X No path
C → B: X No path

BFS EXERCISE OUTPUT
=====

[5] BFS after adding G:
Order: ['A', 'B', 'C', 'D', 'E', 'F', 'G']

[6] Actual paths:
A → F: A → C → F
A → E: A → B → E
A → G: A → C → F → G
F → A: No path
C → B: No path

[7] BFS levels:
Level 0: ['A']
Level 1: ['B', 'C', 'D']
```

```
Level 2: ['E', 'F']  
Level 3: ['G']
```

8. Discussion of the Output

From the before output, BFS visited the nodes in the order A, B, C, D, E, and F. This shows that BFS works level by level, because it first visited A, then its direct neighbors, and only after that moved to the next level. In the after output, node G appears after F because we added the edge $F \rightarrow G$. The program also showed actual paths such as $A \rightarrow F$ and $A \rightarrow G$, and it grouped the nodes into levels, for example Level 0, Level 1, and Level 2. This made it easier to understand the shortest path and how BFS explores the graph.

9. Conclusion

In conclusion, this lab helped us understand BFS both manually and in code. We learned that BFS uses a queue, visits the nearest nodes first, and is useful for finding shortest paths in an unweighted graph. Adding node G helped us see how the traversal changes when the graph is extended, and the path and level outputs made the algorithm much easier to understand.

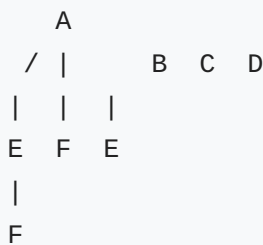
DEPTH-FIRST SEARCH (DFS) LAB REPORT

1. Introduction

In this lab, we worked with Depth-First Search (DFS) and used the given graph to see how DFS moves through nodes. We learned that DFS starts at one node and keeps going deeper along one path before coming back to try another path. Unlike BFS, which moves level by level, DFS follows one branch first and then backtracks when it reaches a dead end.

2. Graph Used in the Lab

The graph used in the DFS lab was the same one from the BFS lab. It can be drawn in a simple layout like this:



Adjacency list: $A \rightarrow [B, C, D]$, $B \rightarrow [E]$, $C \rightarrow [F]$, $D \rightarrow [E]$, $E \rightarrow [F]$, $F \rightarrow []$

3. Manual DFS Trace

Starting from A, the DFS order in this program is affected by the stack. Since the neighbors B, C, and D are pushed in that order, D is popped first. So the manual trace is:

$A \rightarrow D \rightarrow E \rightarrow F \rightarrow \text{backtrack} \rightarrow C \rightarrow \text{backtrack} \rightarrow B$

Therefore, the final DFS visit order is: **[A, D, E, F, C, B]**.

4. Before and After Code Changes

Before the exercise changes, the graph ended at F and the code used the original DFS functions only.

```
graph = {
    'A': ['B', 'C', 'D'],
    'B': ['E'],
    'C': ['F'],
    'D': ['E'],
    'E': ['F'],
}
```

```
'F': []  
}
```

After the exercise changes, we added node G with an edge $F \rightarrow G$. We also added a safer topological sort that checks for a cycle before sorting.

```
graph = {  
    'A': ['B', 'C', 'D'],  
    'B': ['E'],  
    'C': ['F'],  
    'D': ['E'],  
    'E': ['F'],  
    'F': ['G'],  
    'G': []  
}  
  
# topological sort now checks for a cycle first  
def topological_sort_safe(graph):  
    if dfs_has_cycle(graph):  
        print("ERROR: cycle detected, cannot sort")  
        return None  
    return topological_sort(graph)
```

5. Explanation of the Changes

The first change was adding node G after F. This made the DFS path longer because once the algorithm reached F, it could continue to G before backtracking. The second change was improving topological sort so that it first checks whether the graph has a cycle. If there is a cycle, the program prints an error instead of giving an invalid ordering.

6. Before Output Screenshot

This is the output from the original DFS program before the exercise modifications were added.

```
Step 2 Stack (top bottom): ['D', 'C', 'B']  
✓ Visiting 'D' | Path so far: ['A', 'D']  
→ Pushed onto stack: ['E']  
  
Step 3 Stack (top bottom): ['E', 'C', 'B']  
✓ Visiting 'E' | Path so far: ['A', 'D', 'E']  
→ Pushed onto stack: ['F']  
  
Step 4 | Stack (top bottom): ['F', 'C', 'B']  
✓ Visiting 'F' | Path so far: ['A', 'D', 'E', 'F']  
→ Dead end will backtrack
```



```

Step 5 Stack (top bottom): ['C', 'B']
✓ Visiting 'C' | Path so far: ['A', 'D', 'E', 'F', 'C']
→ Dead end will backtrack

Step 6 Stack (top bottom): ['B']
✓ Visiting 'B' | Path so far: ['A', 'D', 'E', 'F', 'C', 'B']
→ Dead end will backtrack

DFS complete! Final order: ['A', 'D', 'E', 'F', 'C', 'B']
=====

[5] Cycle detection:
Original graph has cycle: False
Modified graph (F→A) has cycle: True

[6] Topological sort (DFS finish times):
Result: ['A', 'D', 'C', 'B', 'E', 'F']
Verify: every edge AB should have A appear before B

```

7. After Output Screenshot

This is the output after we made the required changes. It now includes node G and the updated topological sort with cycle detection.

```

✓ Visiting 'E' | Path so far: ['A', 'D', 'E']
→ Pushed onto stack: ['F']

Step 4 | Stack (top bottom): ['F', 'C', 'B']
✓ Visiting 'F' | Path so far: ['A', 'D', 'E', 'F']
→ Pushed onto stack: ['G']

Step 5 | Stack (top bottom): ['G', 'C', 'B']
✓ Visiting 'G' | Path so far: ['A', 'D', 'E', 'F', 'G']
→ Dead end will backtrack

Step 6 Stack (top bottom): ['C', 'B']
✓ Visiting 'C' | Path so far: ['A', 'D', 'E', 'F', 'G', 'C']
→ Dead end will backtrack

Step 7 Stack (top bottom): ['B']
✓ Visiting 'B' | Path so far: ['A', 'D', 'E', 'F', 'G', 'C', 'B']
→ Dead end will backtrack

=====
DFS complete! Final order: ['A', 'D', 'E', 'F', 'G', 'C', 'B']
=====

```

```
[5] Cycle detection:
Original graph has cycle: False
Modified graph (F→A) has cycle: True

[6] Topological sort with cycle detection:
Result: ['A', 'D', 'C', 'B', 'E', 'F', 'G']

[7] Testing topological sort on a cyclic graph:
ERROR: cycle detected, cannot sort
```

8. Discussion of the Output

From the before output, DFS visited the nodes in the order A, D, E, F, C, B. This shows how DFS keeps going down one path first before coming back. In the after output, node G appears after F because we added the edge **F** → **G**. The cycle detection part also worked correctly: the original graph had no cycle, while the modified cyclic graph returned True. The safe topological sort also behaved correctly by printing an error when a cycle was present.

9. Conclusion

In conclusion, this lab helped us understand DFS both manually and in code. We learned that DFS uses a stack or recursion, goes deep into one branch first, and then backtracks. We also saw how DFS can be used for cycle detection and topological sorting. Adding node G helped us see how changing a graph affects the traversal order and the final output.